PENGOLAHAN CITRA DIGITAL (3 SKS / 150 MENIT)

Minggu ke-5: Spatial Filtering untuk Noise Reduction & Image Smoothing

Sub-CPMK 5: Menerapkan konvolusi 2D, Mean, Gaussian, Median Filter untuk derau impulsif, dan Bilateral Filter untuk penghalusan yang mempertahankan tepi (edge-preserving).

Endah Kristiani, Ph.D.

Struktur Perkuliahan & Alokasi Waktu (150 Menit)

Sesi 1: Teori Konvolusi 2D, Padding & Filter Spasial (50 Menit)

Aktivitas: Mekanisme geser kernel 2D, penanganan batas (Zero/Replicate/Reflect Padding), Mean vs Gaussian vs Median vs Bilateral Filter.

Sesi 2: Studi Kasus Nyata & Bedah Kode Python (50 Menit)

Aktivitas: Studi kasus citra satelit antariksa, AI beauty filter, live coding cv2.blur, cv2.GaussianBlur, cv2.medianBlur, cv2.bilateralFilter.

Sesi 3: Praktikum Mandiri Mahasiswa & Restorasi (50 Menit)

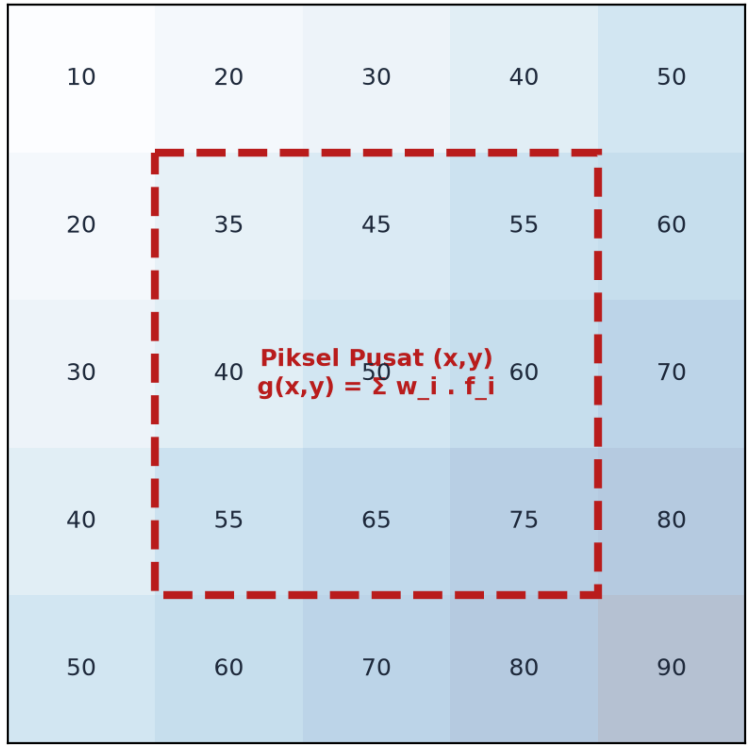
Aktivitas: Injeksi derau Gaussian/Salt-and-Pepper, restorasi filter, dan pengukuran metrik kuantitatif PSNR / MSE.

Mekanisme Konvolusi Spasial 2D & Penanganan Batas

Prinsip Konvolusi Matriks 2D

- Definisi: Operasi penggeseran jendela matriks kernel h(s, t) berukuran m x n di atas setiap piksel citra f(x, y).
- Formulasi Matematis:
  g(x, y) = sum_{s=-a}^a sum_{t=-b}^b h(s, t) . f(x-s, y-t)
- Penanganan Batas Citra (Padding):
  - Zero Padding (BORDER_CONSTANT): Mengisi tepi dengan 0 (hitam).
  - Replicate (BORDER_REPLICATE): Menduplikasi piksel terluar.
  - Reflect (BORDER_REFLECT): Mencerminkan piksel melintasi tepi (standar default terbaik).

Mekanisme Geser Kernel Konvolusi Spasial 2D g(x,y) = h(x,y) * f(x,y)



Contoh Perhitungan Manual Konvolusi 3×3

Misalkan kita memiliki sub-matriks citra 3×3 di posisi pojok kiri atas dengan **Replicate Padding**:

$$\text{Piksel Citra } f = \begin{bmatrix} 50 & 50 & 80 \\ 50 & 50 & 80 \\ 60 & 60 & 90 \end{bmatrix}, \text{Kernel Mean } w = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Langkah Komputasi: Jumlah $= (50 \times 1) + (50 \times 1) + (80 \times 1) + (50 \times 1) + (50 \times 1) + (80 \times 1) + (60 \times 1) + (60 \times 1) + (90 \times 1) = 570$

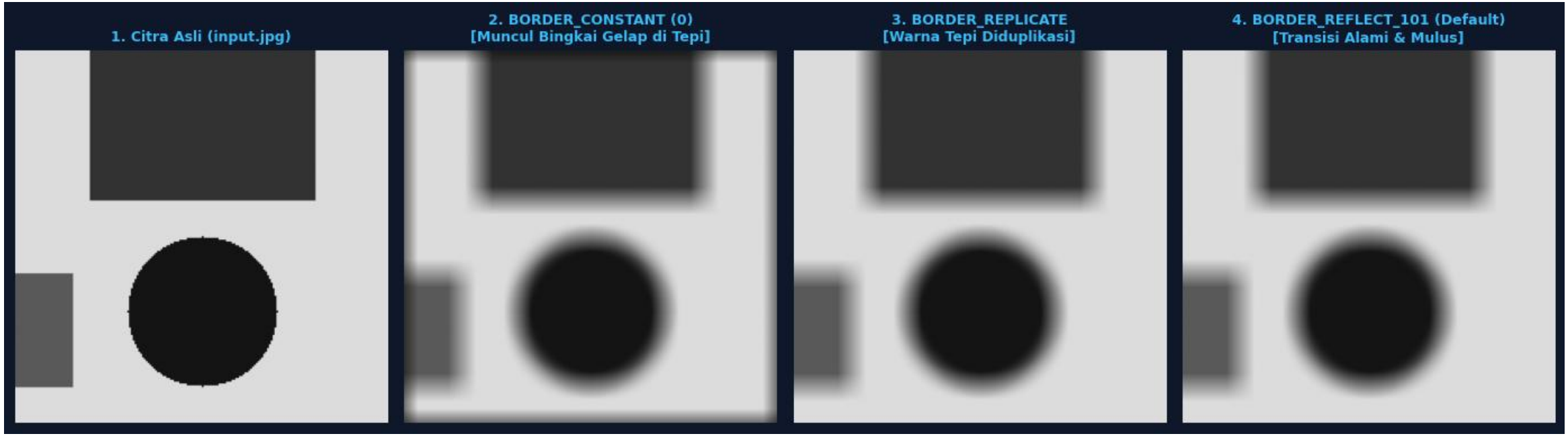
$$\text{Hasil Akhir } g(x, y) = \frac{570}{9} \approx 63.33 \xrightarrow{\text{round}} \mathbf{63}$$

Metode-Metode Penanganan Batas (*Border Padding Techniques*)

Berikut adalah 5 strategi standar penanganan batas yang digunakan dalam pengolahan citra dan OpenCV:

Metode Padding	Cara Kerja	Ilustrasi 1D ([a, b, c, d])	Analisis & Dampak Visual	OpenCV Constant
1. Zero / Constant Padding	Menambahkan nilai konstan 0 (hitam) di luar batas citra.	0 0 | a b c d | 0 0	Cepat dan sederhana, namun menimbulkan efek bingkai gelap (dark frame artifact) pada tepi gambar.	cv2.BORDER_CONSTANT
2. Replicate / Clamp Padding	Menduplikasi piksel terluar ke arah luar sejauh ukuran radius kernel.	a a | a b c d | d d	Mempertahankan kontinuitas warna tepi. Bagus untuk citra dengan latar tepi yang seragam.	cv2.BORDER_REPLICATE
3. Reflect / Mirroring (101) (Rekomendasi / Default)	Mencerminkan piksel di dalam batas ke arah luar tanpa menduplikasi piksel batas terluar.	c b | a b c d | c b	Metode terbaik & standar industri: menghasilkan transisi gradien yang sangat mulus tanpa artifak garis.	cv2.BORDER_REFLECT_101 (BORDER_DEFAULT)
4. Periodic / Wrap Padding	Mengulang citra secara periodik (tepi kiri menyambung ke tepi kanan).	c d | a b c d | a b	Digunakan khusus saat melakukan transformasi domain frekuensi (2D-FFT). Tidak cocok untuk filter spasial biasa.	cv2.BORDER_WRAP
5. Valid / Crop (No Padding)	Konvolusi hanya dihitung pada piksel yang kernelnya masuk penuh di dalam citra.	Output mengecil	Tidak ada padding, namun ukuran citra output menyusut dari $M \times N$ menjadi $(M - 2a) \times (N - 2b)$.	padding='valid

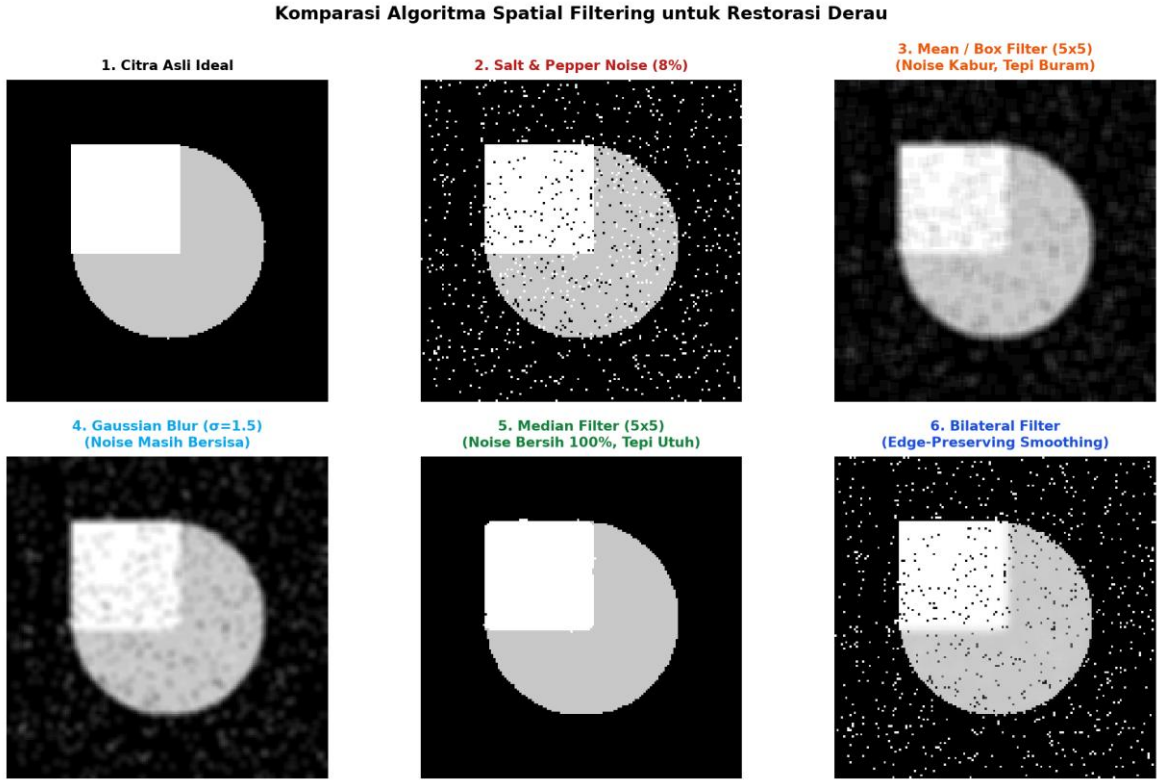
```
import cv2
import numpy as np
# 1. Membaca citra masukan
img = cv2.imread('input.jpg', cv2.IMREAD_GRAYSCALE)
# 2. Mendefinisikan kernel penajam (Sharpening Kernel 3x3)
kernel = np.array([[ 0, -1, 0],
[-1, 5, -1],
[ 0, -1, 0]], dtype=np.float32)
# 3. Eksekusi konvolusi 2D dengan pilihan penanganan batas:
# Opsi A: Menggunakan Reflect 101 (Default & Terbaik)
out_reflect = cv2.filter2D(img, ddepth=-1, kernel=kernel,
borderType=cv2.BORDER_REFLECT_101)
# Opsi B: Menggunakan Constant Zero Padding
out_zeros = cv2.filter2D(img, ddepth=-1, kernel=kernel,
borderType=cv2.BORDER_CONSTANT)
# Opsi C: Menggunakan Replicate Padding
out_replicate = cv2.filter2D(img, ddepth=-1, kernel=kernel,
borderType=cv2.BORDER_REPLICATE)
```


1. Citra Asli (input.jpg)
2. BORDER_CONSTANT (0)
[Muncul Bingkai Gelap di Tepi]
3. BORDER_REPLICATE
[Warna Tepi Diduplikasi]
4. BORDER_REFLECT_101 (Default)
[Transisi Alami & Mulus]

Komparasi Filter: Mean vs Gaussian vs Median vs Bilateral

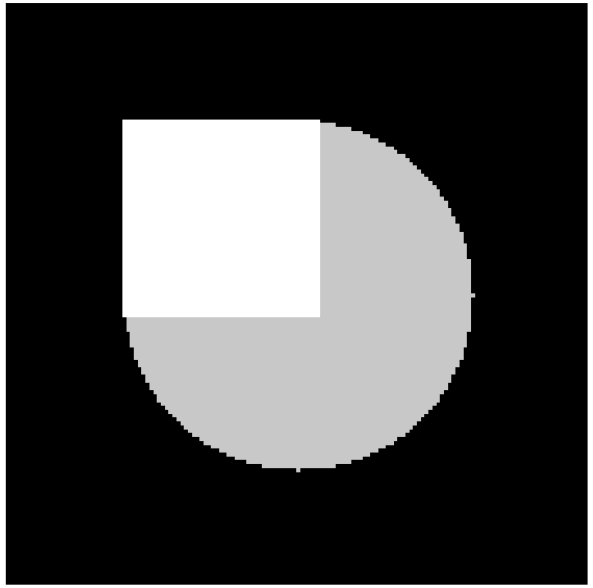
Karakteristik 4 Filter Utama

- 1. Mean Filter (Box): Rata-rata sederhana. Mengaburkan noise tetapi membuat tepi objek buram/kabur.
- 2. Gaussian Blur: Bobot berbasis distribusi lonceng normal. Menghaluskan noise alami dengan gradasi lembut.
- 3. Median Filter: Mengurutkan nilai piksel dan mengambil nilai tengah. STANDAR EMAS untuk Salt-and-Pepper Noise tanpa mengaburkan tepi!
- 4. Bilateral Filter: Menggabungkan bobot spasial dan radiometrik. Menghaluskan permukaan sekaligus MEMPERTAHANKAN TEPI TAJAM (Edge-Preserving).

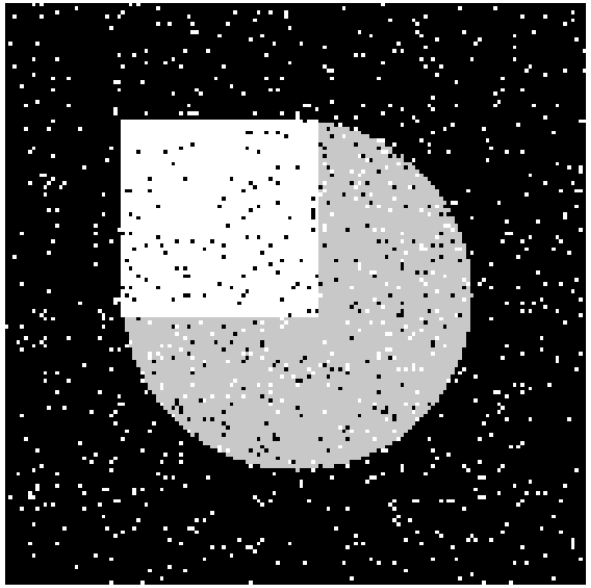


Komparasi Algoritma Spatial Filtering untuk Restorasi Derau

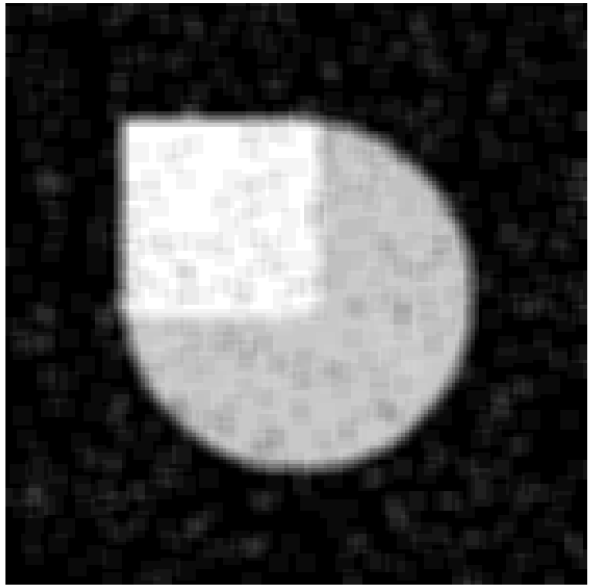
1. Citra Asli Ideal



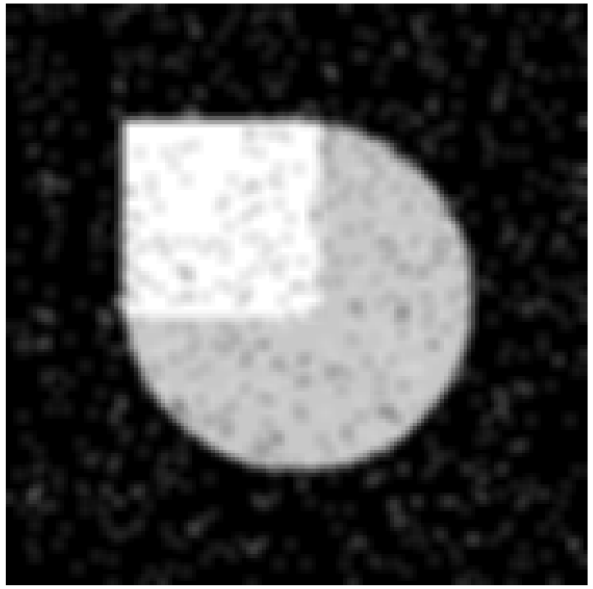
2. Salt & Pepper Noise (8%)



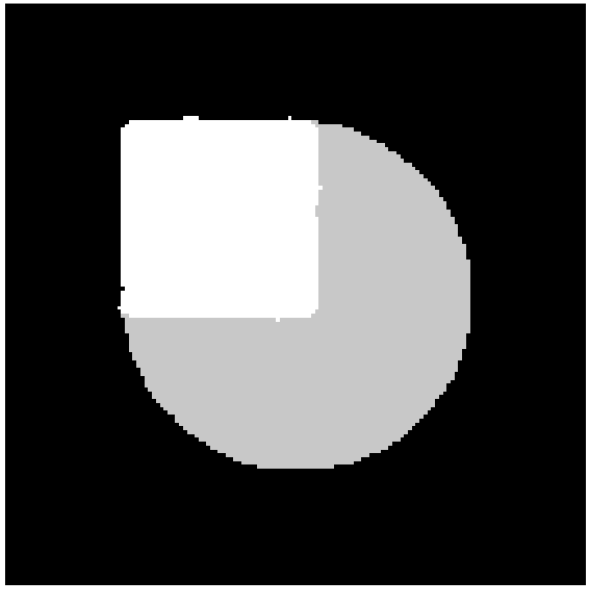
3. Mean / Box Filter (5x5)
(Noise Kabur, Tepi Buram)



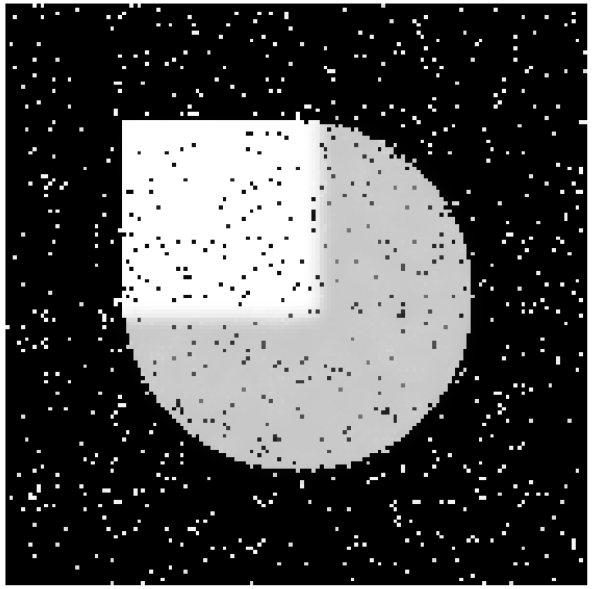
4. Gaussian Blur (σ=1.5)
(Noise Masih Bersisa)



5. Median Filter (5x5)
(Noise Bersih 100%, Tepi Utuh)



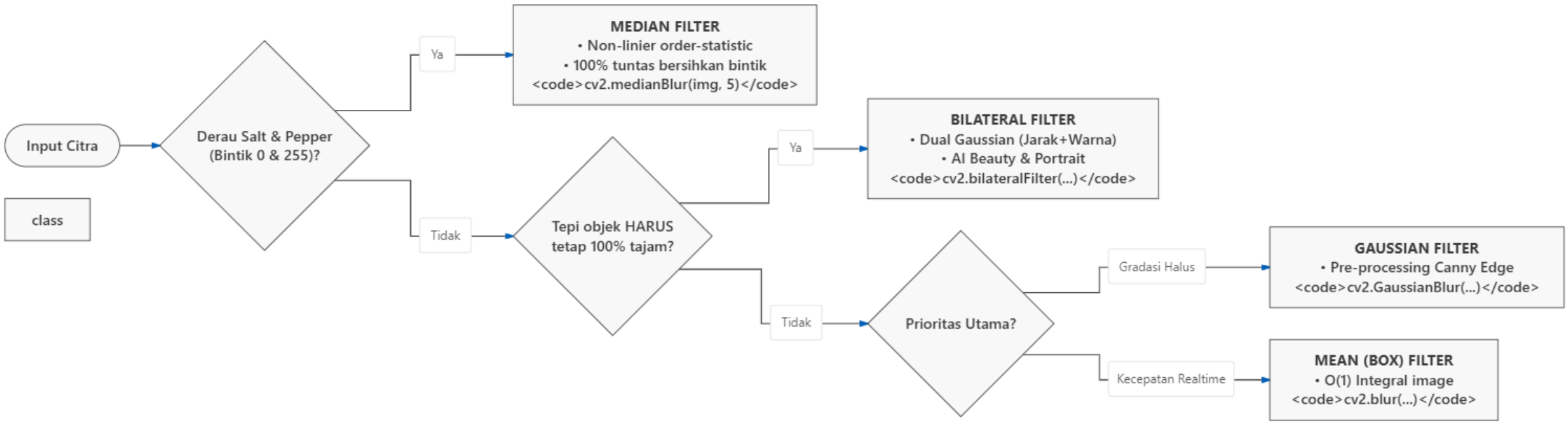
6. Bilateral Filter
(Edge-Preserving Smoothing)



Matriks Perbandingan

Parameter Evaluasi	Mean Filter	Gaussian Filter	Median Filter	Bilateral Filter
Jenis Algoritma	Linier (Averaging)	Linier (Weighted Normal)	Non-Linier (Sorting)	Non-Linier (Dual Gaussian)
Karakteristik Blur	Kasar & mengaburkan	Lembut & natural	Halus pada area homogen	Halus dengan tepi tajam (Edge-preserving)
Penanganan Garis Tepi	Rusak / Sangat kabur	Rusak / Kabur bergradasi	Terjaga baik	Sangat Terjaga Sempurna
Derau Gaussian (Sensor)	Cukup	Sangat Baik	Kurang optimal	Sangat Baik
Derau Salt & Pepper	Gagal (Noda kotor)	Gagal (Noda kotor)	Terbaik (100% Bersih)	Gagal / Kurang optimal
Beban Komputasi	Sangat Rendah ($O(1)$)	Rendah ($O(k)$)	Sedang ($O(\log k)$)	Paling Berat

Flowchart Pemilihan Filter



1. MEDIAN FILTER	2. BILATERAL FILTER	3. GAUSSIAN FILTER	4. MEAN FILTER
Kondisi: Derau bintik hitam-putih (Salt & Pepper)	Kondisi: Menghaluskan tanpa merusak garis tepi tajam	Kondisi: Derau alami sensor kamera / Gaussian Noise	Kondisi: Butuh kecepatan komputasi ekstrem (real-time)
• Non-linier order-statistic • 100% bintik hilang total • Kontur tepi tetap terjaga	• Dual Gaussian (Jarak + Warna) • Menghaluskan kulit homogen • Tepi mata/alis tetap presisi	• Bobot kurva lonceng normal • Transisi blur sangat alami • Standar pra-syarat Canny	• Rata-rata aritmatika seragam • Komputasi cepat $O(1)$ • Tepi objek ikut buram
Kasus: Citra Satelit & Scanner	Kasus: AI Beauty Filter & Potret	Kasus: Deteksi Tepi & Visi Mesin	Kasus: Background Blur Cepat
cv2.medianBlur(img, 5)	cv2.bilateralFilter(img, 9, 75, 75)	cv2.GaussianBlur(img, (5,5), 1.5)	cv2.blur(img, (5,5))

Studi Kasus 5A (Antariksa): Restorasi Citra Satelit dari Derau Kosmik

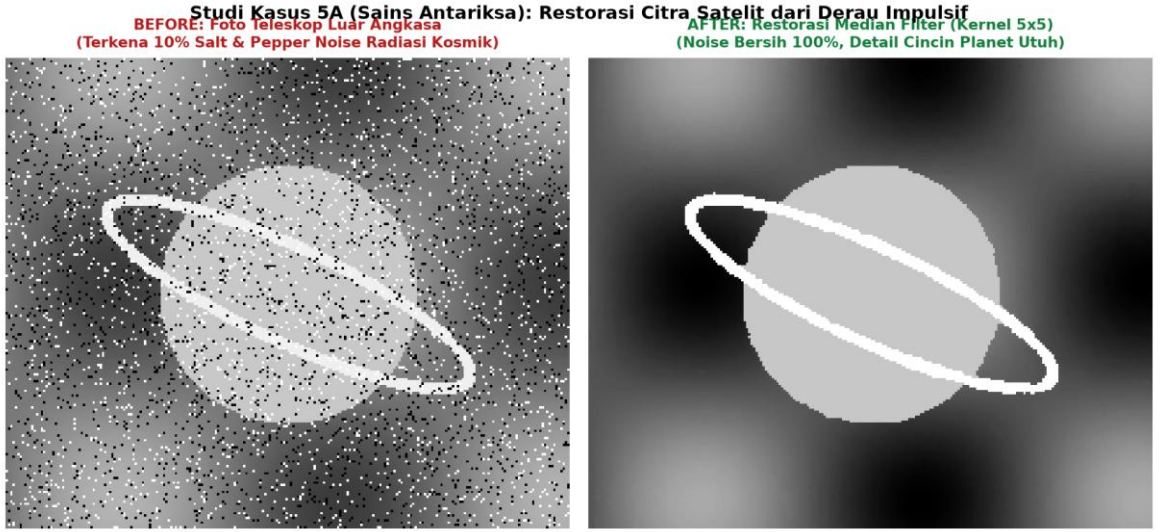
Derau Impulsif Radiasi Kosmik

• Masalah (Before): Sensor teleskop satelit antariksa dihantam partikel radiasi kosmik menghasilkan 10% Salt-and-Pepper Noise (bintik putih-hitam pekat).

• Kelemahan Filter Linier: Mean dan Gaussian gagal karena ikut menghitung rata-rata nilai ekstrim (0 dan 255), membuat citra berkabut.

• Solusi (Median Filter 5x5): Mengurutkan nilai piksel tetangga dan memilih nilai median murni.

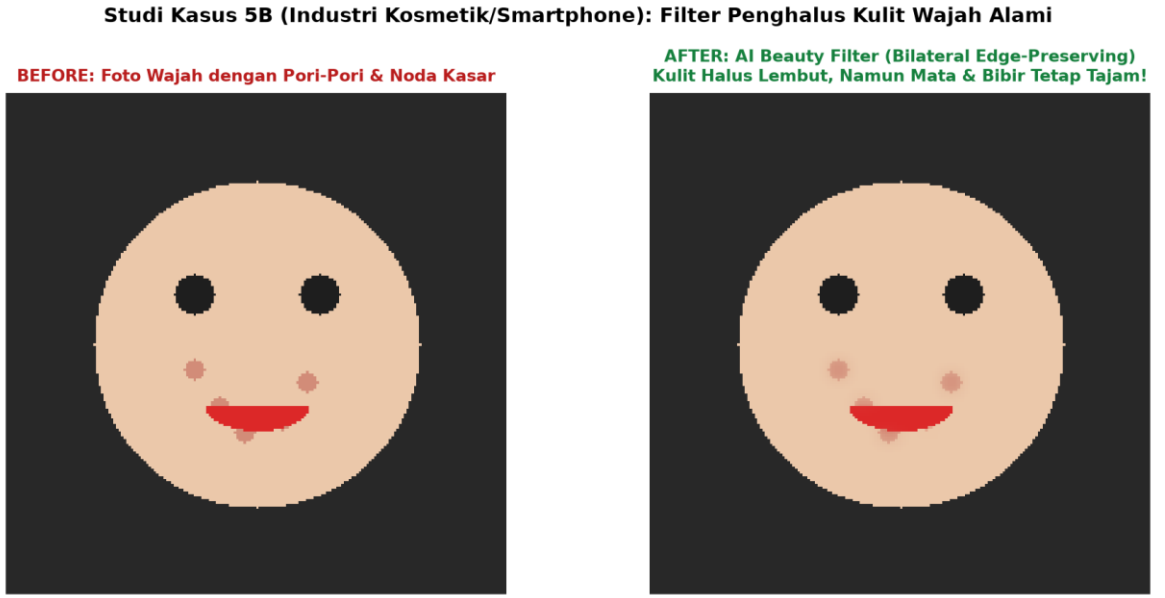
• Hasil (After): Bintik noise bersih 100% tuntas dan garis cincin planet tetap utuh sempurna (PSNR > 36 dB).



Studi Kasus 5B (Smartphone): AI Beauty Filter Kulit Wajah (Bilateral)

Edge-Preserving Skin Smoothing

- Masalah (Before): Filter kecantikan biasa (Gaussian Blur) membuat seluruh wajah kabur, termasuk mata, bulu mata, dan bibir menjadi buram tidak alami.
- Solusi (Bilateral Filter cv2.bilateralFilter):
  - Memadukan bobot jarak spasial (sigmaSpace=85) dan kemiripan intensitas (sigmaColor=85).
  - Hanya menghaluskan area kulit dengan variasi intensitas rendah (pipi, dahi).
- Hasil (After): Tekstur kulit dan noda jerawat menjadi halus lembut, namun garis mata dan bibir tetap tajam alami!



Praktikum 5: Kode Python Spatial Filtering

Fungsi Filter OpenCV

- 1. Box Filter: cv2.blur(img, (ksize, ksize))
- 2. Gaussian Blur: cv2.GaussianBlur(img, (ksize, ksize), sigmaX)
- 3. Median Blur: cv2.medianBlur(img, ksize) (ksize ganjil)
- 4. Bilateral: cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)

Python: spatial_filtering_demo.py

```
import cv2, numpy as np, matplotlib.pyplot as plt

img = cv2.imread('noisy_sample.jpg')

# 1. Mean Filter (5x5)
mean_blur = cv2.blur(img, (5, 5))

# 2. Gaussian Blur (5x5, sigma=1.5)
gauss_blur = cv2.GaussianBlur(img, (5, 5), 1.5)

# 3. Median Filter (ksize=5) -> Ampuh Salt-and-Pepper
median_blur = cv2.medianBlur(img, 5)

# 4. Bilateral Filter -> Beauty Face Smoothing
bilateral = cv2.bilateralFilter(img, 9, 75, 75)

# Visualisasi
fig, axs = plt.subplots(1, 4, figsize=(15, 4))
axs[0].imshow(cv2.cvtColor(mean_blur, cv2.COLOR_BGR2RGB)); axs[0].set_title('Mean 5x5')
axs[1].imshow(cv2.cvtColor(gauss_blur, cv2.COLOR_BGR2RGB)); axs[1].set_title('Gaussian
σ=1.5')
axs[2].imshow(cv2.cvtColor(median_blur, cv2.COLOR_BGR2RGB)); axs[2].set_title('Median
5x5')
axs[3].imshow(cv2.cvtColor(bilateral, cv2.COLOR_BGR2RGB));
axs[3].set_title('Bilateral')
for ax in axs: ax.axis('off')
plt.tight_layout(); plt.show()
```